Reflection as an Interface Description Language: Static and Dynamic Multi-Language Projections of C++ APIs

Frantz Maerten
Look Up Geoscience
fmaerten@gmail.com

August 31, 2026

Abstract

Exposing a C++ library to another language is normally paid for once per target: a hand-written `pybind11` module for Python, an N-API layer for Node, a `sol2` usertype for Lua, a marshalling shim for C#, and a moc-annotated class hierarchy for a Qt user interface. Each of these restates, in a different notation, information the C++ compiler already has.

We present Rosetta, a system that reads that information once — through C++26 static reflection (P2996) over *unmodified* headers — into a target-neutral intermediate representation (IR), and then *projects* that representation two ways. The *static* projection emits ordinary C++ source for nineteen targets; for seventeen of them the emitted code contains no reflection at all and builds with any released C++20 compiler, rather than the experimental fork reflection itself requires. The *dynamic* projection emits the same information as static metadata tables, giving a runtime meta-object model that scripts and user interfaces query by name, without having been compiled against the library's headers.

The central observation is that these are not two features but one mechanism seen from two sides, differing only in *when* names are resolved. We demonstrate this with a measurable consequence: five targets whose binding surfaces are name-keyed — Lua, Node, WebAssembly, C# and Java — must discard C++ overload sets at generation time, yet recover them in full through the dynamic projection, because resolution moves from generation time to call time. The expressiveness difference between targets is therefore not a property of the target language but of its resolution time.

Rosetta additionally treats binding coverage as a first-class artifact: every member that is not exposed is recorded, with a machine-readable reason, in a `coverage.json` that can be diffed across revisions. On a controlled 288-method API, the five name-keyed targets bind 96 methods however many overloads are declared, while the late-bound projection reaches all 288 — two thirds of the interface recovered by deferring resolution alone. Pointed at an unmodified third-party mesh library and asked to bind its entire public namespace unattended, the pipeline reaches 75% of what it is given, and the reason the rest fails is concentrated rather than diffuse: a quarter of that library's declared surface is templates, and 85% of those sit in its vector and matrix value types rather than in the algorithms a caller invokes.

Keywords: static reflection, C++26, language bindings, intermediate representation, meta-object protocol, scientific software.

1 Introduction

A scientific C++ library rarely stays in C++. Its users want it from Python for analysis, from Julia for numerics, from a notebook, from a browser, from behind a REST endpoint, and from a graphical interface that lets them move a slider and see a result. Each of these is, today, a

separate engineering project with its own idioms, its own failure modes and its own maintenance burden — and each one begins by restating facts that the C++ compiler already knows perfectly well: that `Mesh` has a field named `opacity` of type `double`, that `at` is overloaded on `int` and on `int,int`, that `Solver` derives from `Operator`.

That restatement is the cost this paper is about. It is paid N times for N targets, it drifts silently when the C++ API changes, and it is the reason many scientific libraries have exactly one binding — usually Python — and no plausible route to a second.

C++26 static reflection [2] changes what is possible here, because for the first time the information is available *from inside the language*, with the compiler's own answers to the questions that a third-party parser has to re-derive: which overload is hidden by a derived declaration, which special members are implicitly declared, which base is public, what a template specialisation is actually spelled. A system built on it does not need to carry a C++ front-end of its own.

But reflection alone does not produce bindings, and treating it as a better-parser is to under-use it. This paper argues a stronger position:

> *Static reflection can serve as a language-independent interface description mechanism. One reflective pass over an unmodified C++ library yields an intermediate representation from which* both *reflection-free static bindings* and *a persistent runtime meta-object model can be projected — and the two projections differ only in when name resolution happens.*

Contributions.

1. **A reflection-derived, target-neutral IR.** A single `consteval` walk over a type hands each member, together with its full annotation pack, to a visitor. Backends never see `std::meta::info` (Section 5).

2. **A static projection to nineteen heterogeneous targets** — language bindings, documentation, schemas and user interfaces — from that one description (Section 6).

3. **Reflection-free deployment.** Reflection is confined to the generation stage: for seventeen of the nineteen targets the emitted code requires only C++20, so it builds with any released compiler rather than the experimental fork reflection itself needs (Section 4). The two exceptions are deliberate and are stated in Section 12.

4. **A dynamic projection**: the same IR written out as static metadata tables, yielding a runtime meta-object model that a script or a GUI queries by name without knowing the C++ types (Section 7).

5. **Resolution time as the organising axis.** We show that expressiveness lost by early-binding targets is recovered by the late-bound projection, and quantify it (Section 8).

6. **Coverage as a first-class, machine-readable artifact**: what was not bound, and why (Section 9).

What we do not claim. We do not claim that Rosetta produces better Python bindings than a hand-written `pybind11` module; an expert with unlimited time will always win on one target. We claim that the *per-target* cost is what Rosetta removes. Nor do we claim the meta-object model itself is novel — Qt's meta-object system [9], Graphite's GOM [7], Unreal's UObject reflection [3] and Objective-C all long predate it. The novelty is that the model is *derived automatically, by the compiler, from unmodified headers*, and that it shares its description with the static bindings.

2 Motivation: the per-target cost

2.1 One method, five notations

Consider a single member function of a small numerical class — a flat buffer of scalars grouped into items:

Listing 1: The declaration. Everything below is derived from it.

```
namespace serie {
    class Serie {
    public:
        std::size_t itemSize() const; // scalars per item
        // ...
    };
}
```

Listing 2 is what exposing that one function costs in five host environments. Each line is taken verbatim from ROSETTA's own output, but nothing about them is ROSETTA-specific — they are what a developer writes by hand, and what the corresponding hand-written module contains.

Listing 2: The same member function, five times.

```
// pybind11
c.def("itemSize", &serie::Serie::itemSize, "Scalars per item: ...");

// N-API (Node)
props.push_back(This::InstanceMethod<&This::call_method<
                  &serie::Serie::itemSize>>("itemSize"));

// embind (WebAssembly)
.function("itemSize", static_cast<unsigned long (serie::Serie::*)() const>(
                        &serie::Serie::itemSize))

// jlcxx (Julia)
c.method("itemSize", static_cast<unsigned long (serie::Serie::*)() const>(
                       &serie::Serie::itemSize));

// sol2 (Lua)
c["itemSize"] = &serie::Serie::itemSize;
```

Five notations, and every one of them encodes the same three facts: *which C++ entity* is being exposed, *what name* it takes in the host language, and — explicitly in the embind and jlcxx cases, implicitly in the others — *what its signature is*. All three are already known to the compiler that just parsed Listing 1. None of them is a design decision; they are transcription.

The two lines carrying an explicit `static_cast` are worth noticing, because they show the transcription failing in a way that is easy to get wrong: `&Serie::itemSize` names an *overload set*, not a function, so a target that cannot take an overload set has to spell out the signature to pick a member. That spelling must then track the declaration by hand. It is exactly the kind of duplicated fact that drifts.

2.2 The cost is differential, not just initial

Writing five bindings once is a bounded, if tedious, expense. The recurring one is that the C++ API moves. A method added to `Serie` must be added five more times, in five notations, in five files; a renamed parameter, a changed return type or a new overload propagates the same way.

We measure this in Section 10.2: across three targets, hand-writable binding code grows at 67 lines per class, and four generated source files change on *every* version bump of the library. Nothing about the arithmetic depends on the bindings being generated — 67 lines per class is what a developer would otherwise be maintaining.

Worse, the failure mode is silent. A method that nobody remembered to add to the Lua module is indistinguishable, from outside, from a method that was never supposed to be there: the module simply does not have it, and no build step objects. This is the observation that motivates making binding coverage an explicit, machine-readable artifact rather than an inference from what happens to compile (Section 9).

2.3 The second-order problem: interfaces over the same facts

A binding is not the only consumer of a type's structure. A property panel needs to know that `Serie` has an `itemSize`, what type it is, whether it is writable, and what range it may take. A console inspector, a QML view, an immediate-mode debug overlay and a REST schema all need the same. Each is a further restatement of the same facts, and because each targets a different toolkit, each is written separately.

This is not hypothetical: it is what ROSETTA's own repository looked like. Before the dynamic projection existed, five hand-written C++ walkers traversed the same metadata, one per consumer — a console interpreter (446 lines), a Qt property panel (401), a Qt main window (235), an ImGui runtime (331) and a QML reflected object (102): about 1 500 lines expressing one idea five times, each of which had to be revisited whenever the metadata changed shape.

The measure of how much of that was accidental rather than essential: the Qt property panel's 401 lines of C++ become 30 lines of Python once the metadata is reachable from the host language (Section 7). What the C++ walkers were mostly doing was not building widgets — it was re-deriving structure that something else already knew.

2.4 The requirement

Both problems have the same shape. The information is available exactly once, in the compiler, at the moment it parses the header; and it is then re-entered by hand, once per consumer, in a notation that consumer happens to use. So:

> *The description should be extracted once, by the entity that already has it, in a form no consumer's notation is privileged over — and then projected, as many times as there are consumers.*

Static reflection makes the extraction possible. The rest of this paper is about what the intermediate form has to record for the projections to be faithful, and what happens when the consumers disagree about what they can express.

3 Background: C++26 static reflection, and what it forces

P2996 [2] introduces `std::meta::info`, a reflection value denoting a program entity, together with `consteval` queries over it (`members_of`, `type_of`, `identifier_of`, ...) and *splicing*, which turns a reflection value back into a usable declaration.

Two properties matter for what follows.

The compiler's answers, not a parser's. Because the queries run inside a real compilation of the user's headers, they report what the language actually decided. Listing 3 is the canonical case: `Derived::f(double)` hides both base overloads, and a C++ caller without a `using Base::f;` cannot reach them. ROSETTA binds exactly `f(double)`, and records the two hidden declarations as drops. A system built on an independent C++ parser must re-implement name hiding, access control, implicit special-member declaration and template instantiation, and will diverge from the compiler in the corners.

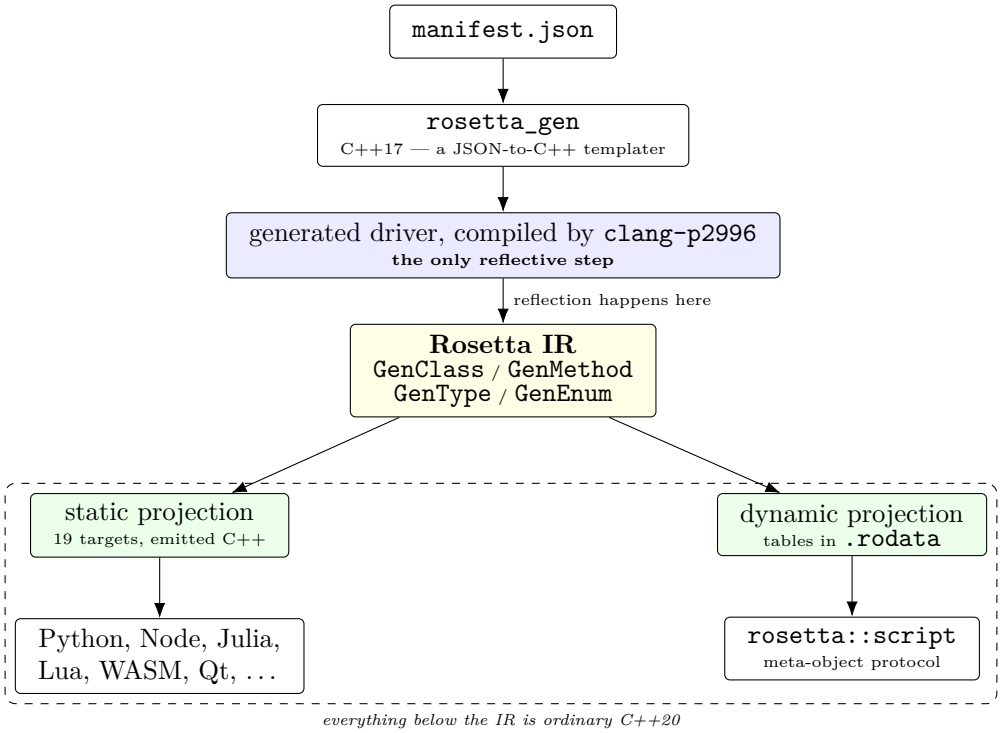

Figure 1: The pipeline. Reflection occurs in exactly one place. The IR is the pivot: to its left the static projection, to its right the dynamic one. Nothing reflection-flavoured survives into the emitted artifacts.

Listing 3: Name hiding is a compiler decision, not a syntactic one.

```
struct Base { int f(int) const; int f(int, int) const; };
struct Derived : Base { int f(double) const; };
// Derived binds only f(double); the two Base overloads are recorded
// as ‘hidden_by_derived‘ drops.
```

The constraint that shapes everything else. Reflection answers questions *at the compile time of a program that includes the user's headers*. It cannot be queried from a script, a build system or a configuration file. Therefore, to learn the shape of a type, something must *compile*; and to write the answers to disk, that something must *run*. Compile-then-run is not an implementation choice — it is the only shape this can have, and it is why ROSETTA is a pipeline rather than a command. Section 4 makes that pipeline explicit.

Availability. P2996 is implemented in `clang-p2996`, a research fork. Section 12 states precisely where this dependency bites and where it does not.

4 Architecture: a two-stage pipeline

Stage 0 — the templater. `rosetta_gen` reads a `manifest.json` and emits a C++ driver program, a header that includes the user's types, and a `CMakeLists.txt`. It performs no reflection and builds with any C++17 compiler; it never opens the user's headers, only copies their names into the driver it writes. Its purpose is that what a project author writes is a *manifest*, not a program.

Stage 1 — reflect and emit. The driver is compiled with `clang-p2996` and executed. During its compilation the reflective walk runs; during its execution it writes files. This is the compile-then-run sequence Section 3 showed to be unavoidable.

Stages 2–3 — build and load. The emitted projects are ordinary C++20. They are configured and built with a released compiler and loaded by their host runtimes. Each stage therefore has a different and decreasing language requirement: C++17 to build the templater, C++26 to run the reflective driver, C++20 to build what it produced.

The library is read twice. Table 1 states the property that makes reflection-free deployment possible: the first pass *describes* the types and needs C++26; the second *calls* them and does not.

pass	by	purpose	language required
stage 1	the reflection driver	**describe** the types	C++26
stage 3	the generated binding	**call** them	C++20

Table 1: Why the emitted artifacts outlive the research toolchain. The requirement *drops* between the two passes, which is the whole point: the research toolchain is needed to learn the shape of a type, not to use it.

This has a practical consequence worth stating early, because it is also a limitation: annotations written *inline* in a user header pull `<experimental/meta>` into that header, so the second pass would need the C++26 toolchain too, and the reflection-free property is lost. Rosetta therefore supports *out-of-line* annotations in a JSON side-car, which keep the user's headers plain C++. That mechanism is not a convenience feature; it is what makes the reflection-free claim hold for annotated libraries (Section 12).

5 The reflection-derived intermediate representation

5.1 One walk, many visitors

The reflective surface of Rosetta is a single `consteval` walk. A visitor implements four `consteval` member templates:

Listing 4: The entire visitor concept. `Fld`/`Fn` are `std::meta::info` non-type template parameters; `Anns...` is the member's full annotation pack.

```
v.template field <Fld, Anns...>(name);
v.template method_instance<Fn, Anns...>(name);
v.template method_static <Fn, Anns...>(name);
v.template constructor <Ctor, Anns...>(); // optional, detected by requires
```

Two design decisions in Listing 4 carry most of the architecture.

First, **the walker has no per-annotation entry point**. It forwards the complete annotation pack and lets the visitor decide what it cares about; a backend that understands `range` and one that does not are the same shape. Adding an annotation kind therefore does not touch the walker.

Second, and more importantly, **backends do not see reflection**. By the time a backend runs it is handling ordinary structs — `GenClass`, `GenMethod`, `GenField`, `GenType`, `GenEnum` — with `std::string` members. A backend author writes no `consteval` code and never touches `std::meta::info`. This is what makes nineteen backends tractable, and it is the reason the dynamic projection is possible at all: an IR made of plain data can be written to disk, whereas reflection values cannot.

5.2 What the IR records

A syntax tree records what the source *says*; an intermediate representation records what it *means*, in a form chosen for its consumers rather than for its producer. Beyond the obvious (names, types, parameters, return types, bases, overload sets, annotations), two categories of record are worth singling out, because they are exactly where that distinction bites.

Identity is split. Every class carries both a fully qualified C++ spelling and an *exposed* name. The first is what emitted C++ must write to resolve the entity, including enclosing classes as well as namespaces; the second is what the target language sees, and may be renamed to avoid a collision between two classes that share an unqualified name. Keeping these apart is what lets a single description serve languages with incompatible naming rules.

Semantic properties, not syntax. The IR records whether a class is abstract, default-constructible, publicly destructible, copy-or-move assignable and copy-or-move constructible. These are not readable from the source text — they are compiler conclusions — and each exists because some backend needs it to avoid emitting code that does not compile. A ref-counted class with a protected destructor, for instance, is a hard compile error inside every runtime that destroys what it wraps.

And what was lost. Each class carries the members the walk could not hand to any backend, with a reason: `no_identifier` (operators and conversion functions have no bindable name), `function_template` (no fixed parameter pack to splice), and `hidden_by_derived`. These feed Section 9.

6 The static projection

6.1 Nineteen targets, one description

category	targets
scripting / VM	`python` (pybind11), `nanobind`, `lua` (sol2), `node` (N-API), `julia` (jlcxx)
web	`wasm` (embind), `typescript`, `rest`, `openapi`
managed	`csharp`, `java`
user interface	`qt`, `qml`, `imgui`, `paraview`
description	`markdown`, `html`, `json`
runtime	`dynamic` (Section 7)

Table 2: The nineteen backends. Their heterogeneity — a Python module, a REST route table, an OpenAPI schema and a Qt widget row are not variations on a theme — is the evidence that the IR is genuinely target-neutral.

The number itself is not a contribution; it is evidence. What matters is that targets in different *categories* consume the same IR: a REST backend turns a method into a URL route, a Qt backend turns a field into a widget row with a slider whose bounds come from a `range` annotation, and a Markdown backend turns a class into documentation — none of them sharing anything but the IR.

6.2 Idiom, not mechanical transliteration

A recurring design question is whether a C++ type should cross the boundary as itself or as the target language's equivalent. ROSETTA's position is the latter, wherever the target has an

equivalent: a sequence should reach the host as the host's own sequence. `std::vector<double>` is a Python `list` and — the subject of this section — a JavaScript `Array`. How far each target gets varies with what its binding library allows: sol2 and jlcxx expose C++ containers as userdata, so the Lua backend adds a second, table-accepting overload beside the container one rather than replacing it. The point of stating this as a position rather than an implementation detail is that the straightforward reading of each binding library pushes the other way, and the WebAssembly target is where that pressure is strongest.

What the library offers. embind gives `std::vector<T>` no conversion at all out of the box; the documented remedy is `register_vector<T>`, which makes it a *bound class*. The consequences are visible in every line of caller code: JavaScript has to construct an `M.vector_double` and fill it by `push_back`, cannot pass an array literal, receives a handle rather than an `Array` on the way out, and must `.delete()` that handle. Listing 5 contrasts the two against the same C++ declaration — the `Serie` of Listing 1, whose constructor takes a `std::vector<double>` and whose `scalars()` returns a `const std::vector<double>&`.

Listing 5: The same two C++ signatures, seen from JavaScript. Above: what `register_vector` yields. Below: what ROSETTA emits.

```
// with register_vector<double> - a sequence as an opaque bound class
const buf = new M.vector_double();
for (const x of [1.0, 2.0, 4.0]) buf.push_back(x);
const s0 = new M.Serie(buf, 1);
buf.delete();
const out = s0.scalars(); // an M.vector_double handle
console.log(out.get(1)); out.delete();

// what rosetta emits - the sequence is the host language's own Array
const s = new M.Serie([1.0, 2.0, 4.0], 1);
console.log(s.scalars()[1]); // a JS Array
```

The upper form is not wrong, and a hand-written binding would reasonably settle for it. It is, however, the one place in ROSETTA where a sequence was not the host language's own array type — an idiom break that a user meets on their first line of code, and one that no other target imposed.

What it takes to close. The fix is a single specialization of embind's own `BindingType` for `std::vector<T>`, putting it on the `emval` wire — the same technique `<emscripten/val.h>` already uses for `std::optional`. Outbound is `val::array(v)`, which memcpy's through a typed-array view when `T` is arithmetic and walks element-wise otherwise; inbound is `vecFromJSArray<T>`. One `register_type<...>()` per element type then declares that type id an `emval` passthrough on the JavaScript side; without it the module aborts at the first call with `BindingError: Unbound type`.

Two properties are what make this worth doing in a generator rather than by hand. First, it composes with the library instead of around it: embind's own `BindingType<const T>` and `BindingType<T&>` forwarders carry the one specialization to `const std::vector<T>&` parameters and returns, so an ordinary member-function pointer still binds — there is no per-method adapter to emit — and nested `vector<vector<T>>` falls out by recursion. Second, it is written once and applies to every class the IR carries. The IR says which members traffic in sequences and what their element types are; the backend decides, once, what a sequence *is* in JavaScript. That asymmetry — one decision, every class — is the leverage the IR buys, and it is the same shape of leverage elsewhere: `std::filesystem::path` marshalled as a string, or an out-parameter joining the return value — a tuple in Python, multiple returns in Lua, an array in JavaScript.

The evidence. `examples/plain-binding` drives one C++ class from four hosts with one driver script each. After the change, the WebAssembly driver is the Node driver line for line except for the two things embind genuinely does differently — the module loads asynchronously and hands out object handles the caller must `.delete()`; and a C++ throw arrives as an emscripten `CppException` rather than as a `TypeError`. Sequences are no longer among the differences, and the two runs print the same values, that one exception line aside. That residue is the honest measure of the position: what is left over is what the target really imposes, not what its binding library found convenient.

Idiom-matching is not free, and it is not universal. It is paid per backend and per type family, in backend code that has to know the target library well — and it stops where the target cannot follow. A `std::vector<T*>` still does not cross this wire, because `val::array` would need an `allow_raw_pointers` policy per element on the way out and `vecFromJSArray` the same on the way in, neither of which the `BindingType` hook can carry. Such a member is skipped with a recorded reason rather than mistranslated (Section 9).

6.3 Where targets differ: overloads

The clearest place the targets diverge is C++ overloading, and it is worth examining because Section 8 turns it into a result.

target	policy	why
`python` (pybind11)	all overloads	repeated `.def` builds one overload set
`nanobind`	all overloads	same model
`julia` (jlcxx)	all overloads	multiple dispatch selects
`qt`	all overloads	one widget row per method; nothing keyed by name
`wasm` (embind)	first only	a second `.function` throws at module init
`node` (N-API)	first only	property descriptors are keyed by name
`lua` (sol2)	first only	`c["at"] = ...` is an assignment
`csharp`, `java`	first only	name-keyed op table; JSON marshalling
`typescript`	first only	describes the N-API module; must not over-promise
`rest`	first only	a route is a URL path
`qml`	first only	`registerInvoker` is keyed by name

Table 3: Per-target overload policy. The split is not about language power — Lua and JavaScript can express dispatch — but about whether the *binding surface* is a name-keyed map. "First" means first-declared, which is stable across regenerations.

Crucially, the IR does not make this decision. It carries the whole overload set, and each backend reduces it if it must, recording every discarded sibling as an `overload_not_expressible` entry in the coverage report. A reduction is a backend policy, never an IR loss — which is precisely why Section 8 can recover it.

7 The dynamic projection: a runtime meta-object model

7.1 Spending the IR on data instead of code

Every backend in Section 6 spends the reflection result on *framework calls*. The information is consumed at generation time, and what survives is code for one specific runtime.

The `dynamic` backend spends it on **data**. It writes the IR back out as static tables — `MetaClass`, `MetaField`, `MetaMethod`, `MetaCtor`, `TypeDesc`, `MetaEnum`, `MetaFunction` — so that the question "what does this class have?" is still answerable at run time, by a caller that was never compiled against the library's headers. The tables live in `.rodata`; the handles that reach them are single pointers, so making the model available to another language copies no metadata.

Access goes through generic thunks: find a class by name, find a member, score an overload set against the actual arguments, get, set, call. Errors are returned as values, never thrown, because unwinding a C++ exception through a foreign VM's frames produces a crash rather than a `TypeError`.

7.2 Pointing Rosetta at itself

The `dynamic` backend makes the metadata available to *C++*. In ROSETTA's own repository, every consumer of it was C++: a console interpreter, a Qt property panel, a Qt main window, an ImGui runtime and a QML reflected object — five hand-written walkers over identical tables, one per toolkit.

The last step removes them, and it is the system's most unusual move: ROSETTA is run *on its own reflection API*. The result is one binding per language of the meta-object protocol itself, after which the walker can be written in the language whose interface it builds. This is the Graphite/GOM manoeuvre [7] — expose the object model to the scripting layer and the GUI becomes a script — reached here by automatic derivation rather than manual registration.

The consequence is the one worth emphasising: **the user's library is not in the binding**. Neither the manifest nor the generated code ever names a class of the target library; a driver names it once, as a string:

Listing 6: The script names the C++ class as data. Nothing in the binding knows `scene::Mesh` exists.

```
m = meta.create("scene::Mesh")
m.set("opacity", 0.5)
m.call("describe", [])
for f in m.cls().fields(): # range -> slider, choices -> combo,
    build_widget(f) # enum -> drop-down, readonly -> disabled
```

So one binding serves *any* library that has been through the `dynamic` backend, and keeps working as that library grows.

7.3 The facade, and why it is necessary

The runtime model cannot be handed to the backends as written, because three of its shapes are unbindable. Table 4 is the reshaping, and it is a useful illustration of a general principle: a meta-object model designed for C++ consumption and one designed to cross a language boundary are not the same model.

in the runtime model	why it blocks	in the facade
`MetaField *fields; size_t n_fields`	pointer-and-count, so vector marshalling never fires	`std::vector<FieldInfo> fields()`
`const MetaClass *`	raw pointer to an unbound type	value-semantic handle
`using Thunk = Any (*)(...)`	function pointers marshal nowhere	`get` / `set` / `call`

Table 4: The facade is this reshaping and nothing more. Each handle wraps one `const` pointer, so the `.rodata` guarantee survives.

7.4 The irreducible hand-written part

Honesty requires naming what reflection cannot do. A host-language value — a Python `float`, a Lua number, a JavaScript `Number` — must be recognised and boxed, and no amount of reflection over C++ knows that these are the same thing. That mapping is per-language knowledge and is written per language.

Only half of it, though. *Reading* a host value is irreducible: only Python knows what `PyFloat_Check` means. *Writing* one has the same shape everywhere, so it lives once in a shared visitor, and each language supplies a sink of nine small methods (`on_none`, `on_bool`, `on_int`, `on_real`, `on_string`, `on_enum`, `on_list`, `on_object`, `on_unknown`). No two languages can then drift apart on whether an enum crosses as its integer.

8 Resolution time as the organising axis

The two projections of Sections 6 and 7 are usually presented, in systems that have both, as unrelated features. We argue they are one mechanism distinguished by a single variable: **when a name is resolved**.

Table 3 showed that five targets — Lua, Node, WebAssembly, C# and Java — must discard C++ overload sets, because their binding surface is a name-keyed map and a second registration under the same name either overwrites the first or fails at module initialisation. This looks like a limitation of those languages. It is not.

Through the dynamic projection, *the same five targets recover the complete overload set*, because `resolve()` scores the whole set against the actual arguments at call time, and can explain a failure when no candidate matches. The overloads were never lost from the IR; they were lost by a projection that had to commit to a name-keyed surface at generation time.

This yields the paper's sharpest claim:

> *The set of C++ members a target can express is not a property of the target language. It is a property of when that target resolves names.*

The same reasoning applies beyond overloads. Annotations are enforced once, in the shared core, rather than re-implemented per backend, so a `range` violation returns the same message in every language — an early-bound projection would have had to re-emit that check nineteen times. Conversely, the price of late binding is a linear name lookup and a scoring pass on every access, which is the right price for menus, property panels, commands and scripting glue, and the wrong price for bulk numerical transfer. Section 10.4 measures it: with the resolution cached the cost falls to 1.2× a direct C++ call, while scoring a three-candidate overload set on every call costs 175×.

	static projection	dynamic projection
resolution	generation time	call time
overloads on name-keyed targets	first only	**complete set**
cost per access	native call	name lookup + overload scoring
new class in the library	regenerate + recompile	regenerate tables only
annotation enforcement	re-emitted per backend	once, in the core
appropriate for	bulk data, hot paths	menus, panels, glue, scripting

Table 5: One IR, two projections, one variable.

Section 10.1 measures this. On a controlled library of 288 methods, the five first-only targets bind 96 of them — a count that does not move as the number of overloads per name grows from one to five — while the late-bound projection reaches all 288. Two thirds of the API is recovered purely by deferring resolution.

9 Coverage as a first-class artifact

9.1 A skip that is not silent

ROSETTA's marshalling policy is deliberately conservative: a member a backend cannot represent is skipped rather than bound into something that throws when called. A skipped method is honest — but on its own it is invisible, and indistinguishable from a method nobody asked for.

Every such decision is therefore recorded, and written to a `coverage.json` alongside the generated code, before anything is compiled. Two stages contribute:

- *Reflection-level* drops, which are backend-independent: `no_identifier`, `function_template`, `hidden_by_derived`.
- *Backend-level* drops, keyed by target, because only the backend knows: `unmarshalable_signature`, `unmarshalable_type`, `sequence_not_adaptable`, `overload_not_expressible`, `static_callback_receiver`.

Reasons are stable slugs intended to be matched by tooling, not read as prose, and each carries a human-readable detail and the signature that distinguishes one overload from another. The practical effect is that a diff of `coverage.json` turns "a method silently stopped binding" into a reviewable line in a pull request.

9.2 What it already shows

The artifact is only worth the claim if it survives an API nobody prepared for it. Section 10.5 does that: pointed at the entire public namespace of an unmodified third-party mesh library, with no manifest author involved, the report accounts for 248 requested entities per target and shows 75% of them binding under pybind11 — and 25% under C# and Java. Nothing about that spread is visible from the generated code, which compiles either way. It is visible because the reasons were written down.

Table 6 is the same instrument at a size a reader can check by hand, on the examples shipped with the system.

example	target	bound	skipped
plain-binding	python / node / wasm	14	0
plain-binding	julia	11	3
geom-expanded	python / nanobind / wasm / lua	9	0
geom-expanded	node	8	1
geom-expanded	julia	8	1
geom-expanded	**csharp / java**	**2**	**7**
scriptable-model	python / lua / node / wasm	116	1
scriptable-model	typescript	117	0

Table 6: Measured coverage, from the checked-in `coverage.json` files. Counts include free functions as well as class members.

The most informative row is the worst one. On `geom-expanded`, the C# and Java backends express two entities of nine: their marshalling goes through JSON and a name-keyed operation table, and seven do not survive that. A generator that reported only its successes would show the same green as Python's 9-of-9. Making the failure legible, per target and per member, is what turns "multi-target" from a claim into a measurement — and it is what lets a user choose a target with knowledge of what they are giving up. That the same two backends are also the floor on a real library, for the same reason, is the strongest evidence that the small table is measuring something real rather than an artifact of a toy.

The claim, tested. This section asserts that every member which is not exposed is recorded. That is a falsifiable claim about the implementation, and putting it in front of a real library falsified two parts of it. No backend recorded *free functions* at all — so on a library whose algorithms are free functions over one data structure, the report covered a fifth of the API and silently omitted the rest, bound and skipped alike. And the `typescript` backend recorded nothing it bound, which is why it is absent from earlier versions of Table 6 entirely: a target that logs only skips does not appear in a report keyed by what recorded something. Both are fixed, and the counts above are the post-fix ones. The general point is uncomfortable but is the reason to build the artifact in the first place: a coverage report is itself a measurement instrument, and an instrument nobody tests against an adversarial input records what its author expected rather than what happened.

10 Experimental evaluation

All five experiments below are run and reported. The harness lives in `paper/experimental-eval/`, one directory per experiment, each reproducible with a single `run.sh`.

10.1 E1 — Overload recovery

Design. We generate a synthetic library of C classes $\times$ N method names, where each name carries R overloads, plus one uniquely-named *control* method per name that is never part of an overload set. Every parameter and return type is one that every backend can marshal (`int`, `double`, `std::string`), and every class is default-constructible, publicly destructible and copyable. The intent is that the overload policy is the *only* thing that can cost a member its binding; anything else is a confound and is reported in a separate column rather than folded into the result. We fix $C = 8$, $N = 6$ and sweep $R \in \{1, \ldots, 5\}$.

The experiment needs only the generation stage: `coverage.json` is written before any binding is compiled, so no host toolchain beyond `clang-p2996` is involved. The full sweep runs in under a minute.

Result. Figure 2 and Table 7 report methods bound. The five first-only targets bind a *constant* 96 methods however many overloads the library declares, while the late-bound projection tracks the library exactly. The recovered count is $C \cdot N \cdot (R - 1)$ — precisely the predicted drop, with no residue. At $R = 5$, **192 of 288 methods (66.7%) of the API are unreachable** through the static binding on those five targets, and **all 192 are reachable** through the late-bound one.

overloads per name R	1	2	3	4	5
methods in the library	96	144	192	240	288
`python` / `nanobind` / `julia`	96	144	192	240	288
`lua` / `node` / `wasm` / `csharp` / `java`	96	96	96	96	96
`typescript` (declares `node`'s module)	96	96	96	96	96
`dynamic` (late-bound)	96	144	192	240	288
recovered by late binding	0	**48**	**96**	**144**	**192**

Table 7: E1, methods bound. Two controls were clean in every configuration: zero members were dropped for any reason other than the overload policy, and zero were dropped by the reflective walk before a backend saw them — so every difference above is attributable to a backend decision, not to information loss upstream.

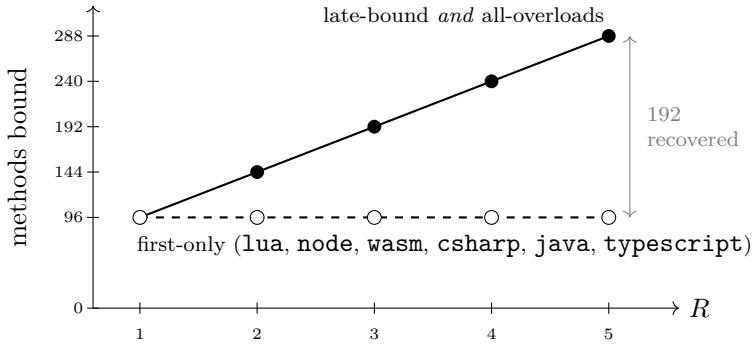

Figure 2: E1: methods bound as the number of overloads per name grows ($C = 8$, $N = 6$). The first-only targets are flat — their binding surface is a name-keyed map, so a name can be registered once regardless of how many overloads exist. The late-bound projection coincides with the library. The overloads were never lost from the IR; they were lost by a projection that had to commit at generation time.

An instrument the experiment found broken. On its first run E1 reported `typescript` as binding zero members in every configuration. The backend recorded its skips but never called `note_bound`, so its coverage figure was structurally zero whatever it emitted — a defect in the instrumentation, not in the binding. The analysis detects this shape (skips > 0 with bound $= 0$) and refuses to average it in, which is how it was caught rather than published. It is now fixed, and the row above is the result: `typescript` tracks `node` exactly, at 96 bound and 192 overload drops at $R = 5$.

That it tracks `node` is the point rather than a coincidence. TypeScript *does* support declaration overloads, so nothing about the language forces the policy; the `.d.ts` describes the N-API module, and declaring an overload the module does not bind would promise the caller a method that is not there. Its first-only policy is inherited from what it describes, which is why it is listed apart from the five whose own binding surface is name-keyed.

Threats to validity. The library is synthetic, and overload multiplicity is controlled precisely because it is the independent variable — E5 measures a real API, where the distribution of overload-set sizes is whatever it happens to be. The result also says nothing about cost: every recovered call pays a name lookup and an overload-scoring pass, which E4 measures. The honest summary is that late binding buys expressiveness with per-call time.

10.2 E2 — The cost of API evolution

Design. One synthetic library in three cumulative versions — 10, 30 and 130 classes, each with four methods and three fields. Versions are strict supersets, so "what changed" is a genuine diff rather than a reshuffle. Three arms are generated for every version: the *static* binding (pybind11, N-API and sol2 together), the *metadata tables* that are stage 1 of the scriptable projection, and the *scriptable binding* of `rosetta::script` that is stage 2.

The static arm doubles as the stand-in for hand-written bindings: the emitted pybind11 module is, line for line, what a developer would otherwise write. We measure the size of that artifact, not the time to produce it — this is an upper bound on what automation removes, not a developer-hours study.

Result. Table 8 reports it. Every relationship is exactly linear over the measured range.

	10 classes	30	130	fit
generated binding source (LOC)				
static binding	763	2 103	8 803	$93 + 67N$
scriptable stage 1 — tables	1 106	3 006	12 506	$156 + 95N$
scriptable stage 2 — the binding	1 208	1 208	1 208	**constant**
human-authored manifest (lines)				
static	53	133	533	$13 + 4N$
scriptable stage 2	25	25	25	**constant**
source files changed by the version bump				
static binding	—	4	4	
scriptable stage 1 — tables	—	2	2	
scriptable stage 2 — the binding	—	**0**	**0**	
generation wall-clock, driver compile (s)				
static	3	5	14	
scriptable stage 2	5	5	**5**	

Table 8: E2. The scriptable binding is byte-identical at 10 classes and at 130, and its manifest never names a library class. Growing the library by 120 classes changes zero of its source files.

The result is not that it generates less code. It generates *more*. Summed across both stages the scriptable arm produces 2 314 LOC at 10 classes and 13 714 at 130, against 763 and 8 803 for the static binding: the metadata tables cost more per class (95 LOC) than the pybind11 registration they replace (67 LOC), and stage 2 adds a fixed 1 208 LOC. Read purely as an artifact-size question, the scriptable projection does not break even until about seventeen classes, and never catches up in total bytes.

What changes is *what has to change, and who writes it*. Three quantities are constant in the size of the library: the binding's source, its manifest, and its generation time. And the fixed 1 208 LOC is not a per-library cost at all — it is a binding of `rosetta::script`, identical for every library that has been through the `dynamic` backend, so it is paid once rather than once per project. The static arm's $67N$ is paid again by every library.

What still scales. The metadata tables regenerate on every version bump and two of their files change each time. The scriptable projection makes the *binding* independent of the library; it does not make the pipeline independent of it, and a claim of $O(1)$ maintenance for the whole chain would be false. Section 12 records this.

Threats to validity. The library is uniform, so LOC per class is a cleaner constant than a real API would give; what is tested is the scaling relationship, not the constants. Only generation is measured — compiling 12 506 lines of tables is not free, and that cost is measured in Section 10.3.

10.3 E3 — Generation cost and scalability

E1 and E2 measure what the pipeline *produces*. E3 measures what running it *costs*, and where that cost sits.

Design. A synthetic library with four independent size knobs — C classes, M method names per class, F fields per class, R overloads per name — giving $C \cdot (F + M \cdot R)$ reflected members. As in E1 every type is marshalable by every backend and every class is copyable, so nothing is skipped and emitted volume is a function of the knobs alone. Five sweeps vary one knob each

from a base point of $C = 16$, $M = 4$, $F = 3$, $R = 1$ and three targets: classes $1 \dots 128$, methods $1 \dots 16$, fields $1 \dots 16$, overloads $1 \dots 5$, and — holding the library fixed — targets $1, 3, 5, 10, 19$.

Four stages are timed separately: `rosetta_gen` (manifest to driver project), the CMake configure, the *driver compile* with `clang-p2996`, and the generator run. Only the driver compile is reflective; the hypothesis is that it dominates. All figures are from one machine (Apple M2 Pro, macOS 15.7, `clang-p2996` at `9ffb96e`) and the noise floor, measured from configurations that should be identical, is 1.9–4.4%.

Result: the reflective compile is the pipeline. It is a median **94%** of wall-clock across all 28 configurations. The other stages do not compete: `rosetta_gen` takes 7 ms, and the generator run — which writes the entire binding tree — takes about 0.2 s and shows *no* dependence on library size, because at these volumes it is process start-up and file I/O. Emitting 15 000 lines of bindings is not the expensive part of generating them.

Result: linear, in every knob independently. Figure 3 pools every configuration that changes the library. The fit is 2.18 s + 13.7 ms per reflected member ($R^2 = 0.987$), and each sweep on its own is linear to $R^2 \geq 0.997$. Because sweeps B, C and D each add exactly one *kind* of member, the pooled slope decomposes:

member added	ms per member	R^2
a field	8.8	0.9991
a method with a distinct name	14.1	0.9975
an overload of an existing name	16.0	0.9994

A method costs about 1.6× a field, and an overload sibling costs slightly more than a fresh name — the walk has to spell out a signature to disambiguate it, which is the same fact that made the `static_cast` lines of Listing 2 awkward to transcribe by hand (Section 2). Peak compiler memory grows from 252 MB at 7 members to 425 MB at 896: enough to notice, not enough to constrain.

The 2.18 s intercept is the more practically relevant half of the fit. It is the fixed cost of `<experimental/meta>`, the ROSETTA headers, and all nineteen backends — which are linked into every driver whatever the manifest asks for, because target selection is a runtime registry lookup. Below roughly 160 members, generation is dominated by a constant.

Result: the walk is per-library, not per-target. This is the claim Section 5 makes architecturally, and it is the one E3 can falsify. Holding the library fixed and asking for 1, 3, 5, 10 and then all 19 targets, the driver compile is 3.60–3.67 s — a 1.9% spread, below the noise floor — while the emitted output goes from 1 582 lines in 6 files to 9 998 lines in 85, and the generator run stays at roughly 0.2 s. *The eighteenth additional target is free at generation time.* One `consteval` walk builds the IR; the backends are ordinary functions that consume it at run time, so the per-target cost is text emission measured in milliseconds. Table 9 gives the numbers.

Result: the metadata tables cost 287 bytes per member. The one thing E3 compiles beyond the driver is `auto_dynamic.cpp`, at `-O2` with the *system* compiler — the tables are stock C++20, which is the point of the dynamic backend. Their footprint is 5 228 B + 287 B per member ($R^2 = 0.991$), split roughly evenly between the descriptor tables with their strings and the emitted per-member call thunks. The largest library measured, 896 members, produces 272 kB. That is the price of the runtime meta-object model in the binary, and Section 10.4 prices it per call.

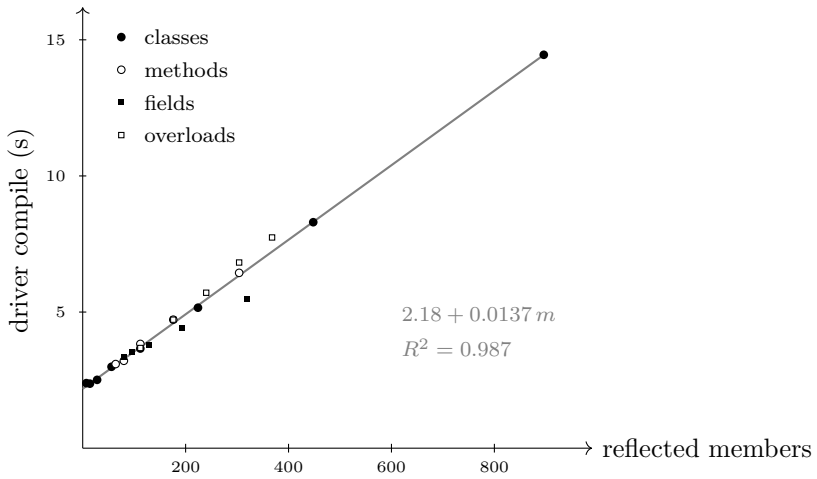

Figure 3: E3: driver compile time against reflected members, pooling the four sweeps that change the library. Whichever knob moved the member count, the cost lands on the same line — the walk is per-member, and no knob is disproportionately expensive. The fields sweep sits slightly below it and the overloads sweep slightly above, which is the per-kind difference reported above.

targets	driver compile (s)	generator run (s)	emitted lines	files
1	3.65	0.16	1 582	6
3	3.67	0.20	2 027	15
5	3.60	0.30	2 460	21
10	3.62	0.17	5 553	58
19	3.64	0.17	9 998	85

Table 9: E3, sweep E. Identical library throughout; only the number of requested targets changes. A 6.3× increase in emitted output costs nothing measurable in reflection time.

That compile is also the cost E2 deferred to here. It runs at $0.51\,\mathrm{s} + 1.32\,\mathrm{ms}$ per member — an order of magnitude cheaper per member than the reflective walk that produced the tables, which is what one would hope from code whose entire design is that it is data rather than reflection.

What this bounds. A 1 000-member library — larger than most single-header scientific APIs — regenerates in about 16 s for as many targets as one cares to name, on one core, and adds a quarter of a megabyte if the dynamic projection is wanted. That is a build step, not a research problem. The honest caveat is that it is *serial*: the driver is one translation unit by construction, so the reflective compile cannot use the other eleven cores of the machine it was measured on.

Threats to validity. The library is uniform, so cost per member is a cleaner constant than a real API would give; the scaling relationship is what is tested, not the constants. One machine and one revision of a research compiler that is not tuned for compile speed: the shape of the curve travels, the seconds do not. And E3 stops after generation, with the single metadata compile as the deliberate exception — *compiling* the emitted bindings costs considerably more than generating them, but that is a cost of pybind11, N-API and embind rather than of Rosetta.

10.4 E4 — Runtime overhead

E1 showed that late binding recovers API surface. E4 prices it.

Design: two layers. Measuring only from a host language would confound two independent costs. Layer A runs entirely in C++ — *direct* call, *dyn* (resolved by name on every call), and *dyn-cached* (the `MetaMethod` resolved once and reused) — and so isolates late binding with no interpreter present. Layer B runs the same operations from Python through the generated pybind11 module and through the scriptable meta-object binding, and so measures late binding *and* the language boundary together. The difference between the layers is the interpreter's share.

The cached arm is not a strawman-avoidance device: `dynamic.h` explicitly recommends it ("the returned pointer is stable for the process, so a wrapper should cache it per call site"), so reporting only the uncached path would price a usage the documentation tells the reader not to write.

	layer A — C++			ratio	
operation	direct	dyn	dyn-cached	dyn	cached
trivial-call	3.6 ns	51.5 ns	4.2 ns	14.4×	**1.2**×
field-get	3.6 ns	10.8 ns	4.2 ns	3.0×	**1.2**×
field-set	3.6 ns	75.2 ns	4.2 ns	21.0×	**1.2**×
3-arg-call	3.6 ns	159.2 ns	14.6 ns	44.4×	4.1×
object-return	3.4 ns	187.7 ns	107.4 ns	54.8×	31.3×
vector-1000	386.6 ns	17.5 µs	7.8 µs	45.4×	20.2×
overload-3way	3.6 ns	629.0 ns	14.2 ns	**175.3**×	4.0×

	layer B — from Python		
operation	pybind11	scriptable	ratio
trivial-call	86.9 ns	467.0 ns	5.4×
field-get	87.6 ns	317.3 ns	3.6×
field-set	92.6 ns	436.9 ns	4.7×
3-arg-call	119.8 ns	895.6 ns	7.5×
object-return	245.0 ns	615.2 ns	2.5×
vector-1000	17.1 µs	42.9 µs	2.5×
overload-3way	108.4 ns	1 216 ns	11.2×

Table 10: E4. Layer A isolates late binding; layer B adds the language boundary. Best of five; repeatability across consecutive runs ±1.5%.

Most of the cost is name lookup, and caching removes it. For the three cheapest operations, caching the resolved member brings late binding to 1.2× a direct C++ call. The thunk is not the problem; finding it is.

Overload resolution is the expensive lookup — and it is the one E1 credits. Scoring a three-candidate set costs 629 ns uncached against 14 ns cached, and is the worst ratio in the experiment at 175× direct. Read with Section 10.1, this is the whole trade in two numbers: late binding recovers two thirds of the API on name-keyed targets, and scoring the candidates at every call is what that recovery costs. Caching the resolution keeps the expressiveness at 4.0×.

Marshalling cannot be cached away. Object return remains at 31× and the 1000-element vector at 20× even fully cached and pre-boxed, because the cost is boxing each value into an

`Any` rather than finding the method. This is the floor of the dynamic model, and no amount of call-site caching moves it.

From a host language the penalty shrinks. Crossing the boundary already costs 87–245 ns, so a 175× C++ ratio becomes 11.2× seen from Python and the bulk-data case falls to 2.5×. The interpreter, not the meta-object model, is the dominant cost of scripting a C++ library.

The rule, in numbers. At 317–467 ns per scriptable operation, a property panel with 100 fields costs roughly 32 µs — imperceptible — while a one-million-element loop costs 0.47 s. `docs/PIPELINE.md` claims in prose that the model is right for menus, panels and glue and wrong for bulk data; the measured boundary is four to five orders of magnitude of call count, not a subtle judgement.

Threats to validity. One machine, one compiler: the ratios travel further than the absolute figures. Microbenchmarks measure microbenchmarks — a real property panel interleaves these calls with layout and I/O that dwarf both arms. The scriptable arm returns an outcome object where the static arm returns a bare float; that allocation is part of the API and is deliberately included rather than subtracted.

10.5 E5 — Coverage on a real library

E1–E4 measure synthetic libraries, which are built to isolate one variable and therefore cannot answer what a real API contains. E5 points the pipeline at one that was never written with binding in mind.

Subject. PMP, the Polygon Mesh Processing library [8], unmodified. It is not a library either author wrote — the relevant objection to using `geogram` here — and it is small enough to bind *exhaustively*, which is the property that matters: at this size the experiment can bind everything and publish the breakdown, rather than choosing a subset and inviting the reader to trust the choice.

Design: the denominator is the difficulty. "What fraction of an API binds" is a measurement only if somebody other than the experimenter decides what the API *is*. The frame is therefore read from clang's own AST: every class, class template, enum and free function declared in namespace `pmp` by a header under `src/pmp/`, excluding the OpenGL viewers. Templates stay *in* the frame — a template cannot be named in a manifest, so filtering them out would raise every percentage below without changing a fact.

Two arms differ in one variable, who chose the entries: *exhaustive*, generated mechanically from that frame, and *curated*, a real hand-written manifest for the same library, retargeted so both emit the same ten backends. The exhaustive arm makes exactly two mechanical concessions, both to the manifest format rather than to taste: an overloaded name carries a signature (`^^pmp::centroid` is ill-formed once the name is declared twice), and classes are topologically sorted so a base precedes its derived classes.

Result: it works, unattended. Nineteen classes, two enums and 91 free functions from an unmodified third-party library reach a driver that compiles and runs with no hand-tuning at all. The build is not error-tolerant by construction: compiling the emitted generator is exactly where a class the walk mishandles would surface as a build failure, which is the outcome Section 9 claims cannot happen, so the harness treats a non-zero exit there as the finding rather than swallowing it. It did not occur.

Result: a quarter of the surface is templates, and they are not spread out. Of 166 declared entities, 120 are nameable and 46 are templates — 28% out of reach before any backend is consulted. But 39 of those 46 are in a single header, `mat_vec.h`, the library's vector and matrix value types; the algorithm headers contribute exactly one. The useful form of the finding is therefore not "templates block a quarter of the API" but that *the templates sit in the value types and the property system, while the algorithm surface a script user actually calls is essentially template-free.* That is a claim about where a reflection-based binder loses, and it is more actionable than the aggregate.

Result: real overload multiplicity is far below E1's sweep. Section 10.1 sweeps overloads-per-name because a synthetic library has no opinion about it. Here the value is measured: PMP's 91 free-function names carry 99 declarations, so a first-only policy costs **8 of 99 (8%)** of the free-function surface, against the two thirds E1 reports at its worst swept case. The mechanism E1 isolates is real and the recovery is real; at free-function scope *this* library barely exercises it. Class members are where the multiplicity actually lives — every first-only target drops 27 members to `overload_not_expressible` — which is the honest shape of E1's result on a real API: significant, and an order of magnitude below the headline the sweep's endpoint suggests.

target	members	functions	fns skipped	bound/req.
`python` / `nanobind` / `julia`	123	64	27	75%
`node`	106	82	9	76%
`lua` / `wasm`	104	64	27	68%
`csharp` / `java`	61	2	89	25%
`dynamic` (late-bound)	141	54	37	73%

Table 11: E5, exhaustive arm. `typescript` is omitted: it records bound fields where the language backends record methods only, so its member column is over a different denominator. The C#/Java collapse is one cause — their boundary is JSON, and 89 of 91 free functions name a type with no JSON marshalling.

Result: curation subtracts reach rather than adding it. The exhaustive arm binds strictly more than the curated one on every target, by 23 to 78 entities. This is worth stating carefully, because the intuitive expectation is the reverse — that a hand-written manifest exists to rescue what automation cannot manage. It does not: it exists to choose. The curated manifest omits property containers, handles and iterators that bind perfectly well and that no script user would call, and in exchange its `out_params` and `sequences` give nicer signatures to the members it does keep. E5 measures reach; the two arms point in opposite directions on reach and quality, and only the first is a number.

A limit that sits above the backends. Eight free-function declarations never reached any backend: two manifest entries binding under one exposed name are a `rosetta_gen` error, so only one overload of a free-function name can be requested at all. This is stricter than the per-target policy of Section 6.3 and it applies to *every* target, including pybind11 and jlcxx, which would have taken the whole set. It is a limitation of the manifest format, not of any target language, and E5 is what surfaced it.

Two instruments this experiment repaired. E5 was not measurable as designed until both were fixed, and both say something about `coverage.json` rather than about PMP. First, *no backend recorded free functions at all* — the report covered class members only, which on a

library whose algorithms are free functions over one data structure meant 99 of 120 nameable entities were invisible to it. Second, the `typescript` backend emitted free-function declarations through no visibility gate, promising callers functions the N-API module does not bind. Both are fixed; the free-function counts above are cross-validated against emitted output (the pybind11 module contains exactly the 64 `m.def` calls the report claims). Section 9 asserts that every unexposed member is recorded with a reason. Before E5 that was true of methods and false of free functions.

Threats to validity. One library, and a well-behaved one: PMP is modern, clean C++17, and a template-heavy or macro-heavy API would give different constants and possibly a different shape. The frame excludes `viewers/`, which needs an OpenGL context — the one judgement in it, and recorded in the output. Coverage counts entities rather than usefulness, which is what makes the curation comparison a statement about reach only. And the member column is not comparable across all targets, for the field-recording asymmetry noted under Table 11.

11 Case study: one library, six consumers, one new class

Sections 10's experiments each isolate one variable. This section does the opposite: it takes the whole pipeline as a user meets it, and asks what happens when the library moves.

11.1 The library and its consumers

`scene.h` is 233 lines of ordinary C++ — no ROSETTA header, no attribute, no base class. Every `doc`, `range`, `combobox` and `readonly` hint lives out of line in JSON side-cars named by the manifest, so the header is what an unmodified third-party library looks like. It declares a value type (`Vec3`), a document (`Mesh`, carrying real geometry behind `positions()`/`triangles()`), and an enum.

One `dynamic` projection of it feeds six consumers, none of which is compiled against `scene.h` and none of which names a bound type:

- a terminal interpreter and a registry inspector, both in C++;
- a Qt viewer — 3D view, generated property panel, completing console;
- Python, Lua and Node drivers, and a `three.js` browser editor over WebAssembly, all going through the *scriptable* projection: a binding of `rosetta::script` itself, which names no library class at all (Section 7).

Why that second group matters. The C++ consumers each contain a hand-written walker over the metadata — `interp.h`, the property panel, the Add menu, and elsewhere in the tree an ImGui and a QML one: five walkers, one per toolkit, all traversing the same tables. Binding the reflection API instead moves that walk into the language whose interface it builds, so the Qt panel's dispatch is written in Python where the Qt binding already lives. We resist the tempting line count here: the Python spec-builder is 27 lines against the C++ panel's 401, but the C++ file also constructs and wires the widgets, which the Python one hands to PyQt. The defensible statement is structural, not a ratio — *one walker per toolkit becomes one walker per toolkit in that toolkit's own language*, and the walker stops being C++.

11.2 Growing the library

The library then gains a third kind of thing: a *solver*. `scene::Relaxer` improves the shape of a triangle mesh by tangential Laplacian relaxation — each pass moves every vertex a fraction `step` toward the centroid of its one-ring, which equalises edge lengths and so drives triangles

toward equilateral. The displacement is projected onto the vertex tangent plane, which is what stops the classic collapse of naive Laplacian smoothing: on the Stanford bunny at 40 passes, the bounding-box diagonal loses 1.05% with that projection off and 0.01% with it on, for the same quality gain.

A solver is the interesting case for this projection because it is mostly *tuning*: four public parameters with declared ranges, three read-only outputs, and one method that runs for a while and mutates something else. That is exactly a property sheet, and nobody writes one.

	mean quality	worst triangle	bbox diagonal
bunny, before	0.834	0.014	1.6072
bunny, after 25 passes	**0.912**	**0.102**	1.6076

Table 12: The solver, driven through the dynamic model. Quality is $4\sqrt{3}A/(a^2+b^2+c^2)$, which is 1 for an equilateral triangle. The worst triangle improves by 7× while the silhouette does not move. Identical figures are produced from the C++ console and from Python, because both call the same tables.

What it cost downstream: nothing. Adding the solver touched three files, all of them the library's own — `scene.h`, one annotation side-car, and one line of the manifest naming the new class. No consumer changed: not the Qt viewer, not the property panel, not the console, not the terminal interpreter, and none of the five scriptable drivers. Yet the generated inspector lists the solver's fields with their ranges and read-only flags, the console runs it (Figure 4), and the Python driver's property sheet grows its rows.

11.3 The claim, checked rather than asserted

A case study can assert anything, so this one is also a test (`E6-case-study/`). The same library goes through the pipeline twice, differing only in whether the manifest lists `Relaxer`, and the generated artifacts are compared. The prediction is deliberately two-sided, because a byte-identity test invites a vacuous pass:

- the *metadata* must differ — 4 stage-1 files change, and `Relaxer` is present in the "after" tables and absent from "before";
- the *binding* must not — and 0 of 18 stage-2 files change, across 4 173 generated lines.

Both hold. That second number is the point of Section 7 stated as an artifact: the binding a host imports is a binding of the reflection API, so the library grew and the thing every consumer depends on did not move a byte. E2 measures the same property as a file-count across a 13× growth; this measures it as byte-identity across one concrete, human-sized change, and fails the run if it ever stops being true.

What this does not show. The harness compares generated artifacts; that the hand-written consumers still work rests on the version-control evidence above, which is weaker. The library is ours — a demonstration library small enough for a reader to hold in their head — so the third-party evidence stays in Section 10.5. And the outline for this section once promised a Julia arm; the *scriptable* projection ships value casters for Python, Lua, Node and WebAssembly but not Julia, so that arm does not exist and is not claimed.

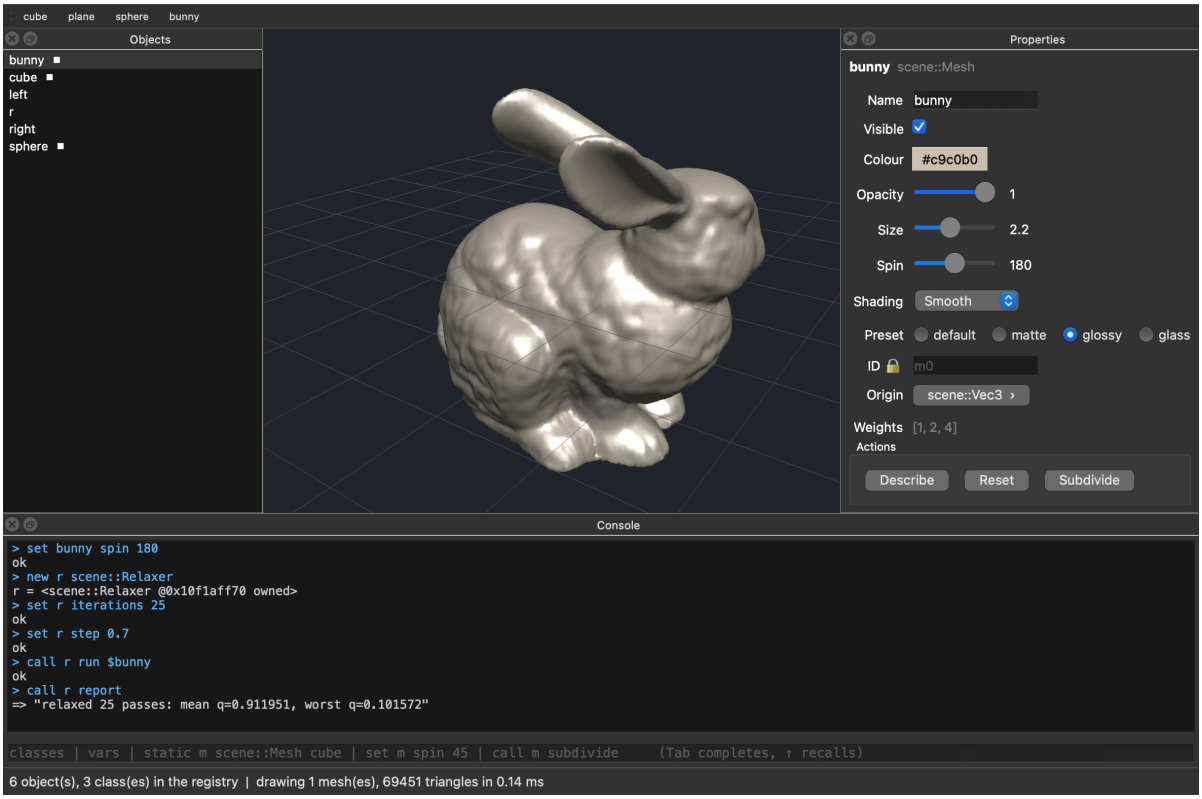

Figure 4: The Qt viewer after the library grew a solver. The console session creates a `scene::Relaxer`, sets its parameters and calls `run` on the bunny; the object list gains `r`; the view redraws from the mutated mesh. Nothing in `qt/` was recompiled against the solver, and nothing in it mentions one — the window is built by querying metadata that grew. The property sheet on the right is generated the same way: its rows, their widgets and their bounds all come from annotations travelling with the metadata rather than from code in `qt/` — including the `Preset` row, whose four choices are read from a `combobox` annotation the view never hard-codes.

12 Limitations

1. **Research toolchain.** Stage 1 requires `clang-p2996`. Stages 0 and 2–3 do not, so the emitted artifacts outlive the fork — but a project cannot re-generate without it.

2. **Two targets are not reflection-free.** The claim in Section 4 holds for seventeen of the nineteen backends. The `rest` and `json` backends emit projects whose sources include a *runtime visitor* that still splices reflections when the binding is compiled, and their generated `CMakeLists.txt` accordingly requests C++26. This is a design choice rather than an oversight — reflecting at binding-compile time keeps those two emitters very small, because the per-member code is written once as a template instead of being expanded by the generator — but it is a real trade of deployment simplicity for generation simplicity, and it means those two targets inherit the fork dependency. The distinction is worth naming: a backend may spend the IR either at generation time or at binding-compile time, and only the former yields a reflection-free artifact.

3. **Inline annotations forfeit the reflection-free property.** An annotation written in a user header pulls `<experimental/meta>` into that header, so the calling pass needs C++26 as well. Out-of-line annotations exist precisely to avoid this and are load-bearing, not cosmetic.

4. **Type casters are hand-written.** Reading a host-language value is irreducibly per-language: roughly nine methods per language, plus the reading half.

5. **The scriptable projection is not $O(1)$ end to end.** Only the binding is independent of the library: the metadata tables still regenerate on every change, growing at 95 LOC per class, and the scriptable arm emits *more* total code than the static one at every size we measured (Section 10.2). What is constant is the binding, its manifest and its generation time — not the pipeline.

6. **Dynamic dispatch has a real cost.** Linear name lookup plus overload scoring per access. Inappropriate for inner loops; a resolved method handle is stable for the process and can be cached.

7. **A vestigial type in each module.** Bindability is decided at generation time and cannot know that a type caster will exist later, so the boxed-value type must remain in the bound class list even though nothing ever produces it directly. One unused type per module.

8. **Coverage measures exposure, not correctness.** A bound method can still be wrong; `coverage.json` counts what crossed the boundary, not whether it behaves.

9. **No formal semantics.** The IR is defined by its implementation.

13 Related work

⟨Draft the prose around Table 13; citations to be completed.⟩

system	type discovery	header intrusion	targets	runtime model	coverage
SWIG [1]	own parser	`.i` file	many	no	no
pybind11 [5]	hand-written	none	1	no	—
nanobind [4]	hand-written	none	1	no	—
Qt moc [9]	own parser	`Q_OBJECT`	1	**yes**	no
Graphite/GOM [7]	manual registration	macros	Lua + GUI	**yes**	no
Unreal UHT [3]	own parser	`UCLASS`	Blueprint	**yes**	no
cppyy [6]	Clang JIT	none	1	yes	no
Clang-AST generators	Clang AST	none	1–2	no	partial
Rosetta	**language-native**	**none**	**19**	**yes**	**yes**

Table 13: Positioning. Two columns carry the delta — language-native discovery with no header intrusion, and both projections from one description. Every prior system has at most one of them.

The nearest prior art is not another binding generator but the meta-object systems: Qt's, Graphite's GOM, and Unreal's. Each demonstrated, long before C++26, that a runtime object model can drive scripting and generated user interfaces, and Rosetta's dynamic projection is explicitly a descendant of that idea. What each of them requires is a *separate description*: a macro vocabulary in the header and a bespoke parser, or manual registration calls. Rosetta's contribution against this line is that the description is the C++ source itself, read by the compiler, on headers that need no modification — and that the same description simultaneously produces the static bindings, which the meta-object systems do not attempt.

Against the binding-generator line — SWIG, pybind11, nanobind, Clang-AST generators — the delta is the opposite one. Those systems produce good bindings for one or two targets, but each carries either a C++ front-end that must track the language or a per-class hand-written specification, and none produces a runtime object model. Rosetta does not claim better Python bindings than pybind11 or nanobind; it claims that reaching the nineteenth target should not cost nineteen times the first.

14 Conclusion

⟨Write last.⟩

[TODO: Restate: reflection as an interface description mechanism; one IR, two projections; resolution time as the axis that explains their difference; coverage as a measurable property of an interface. Close on the case study.]

Availability

ROSETTA is available at `https://github.com/rosetta-bindings/rosetta` under the MIT licence. **[TODO: archive a release DOI (Zenodo) before submission]**

References

[1] David M. Beazley. SWIG: An easy to use tool for integrating scripting languages with C and C++. *Proceedings of the 4th USENIX Tcl/Tk Workshop*, pages 129–139, 1996.

[2] Wyatt Childers, Peter Dimov, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. P2996: Reflection for C++26. ISO/IEC JTC1/SC22/WG21 proposal, 2024. INCOMPLETE: pin the revision number and date actually used.

[3] Epic Games. Unreal engine reflection system and the Unreal Header Tool. Unreal Engine documentation. INCOMPLETE.

[4] Wenzel Jakob. nanobind: tiny and efficient C++/Python bindings. `https://github.com/wjakob/nanobind`, 2022.

[5] Wenzel Jakob, Jason Rhinelander, and Dean Moldovan. pybind11 — seamless operability between C++11 and Python. `https://github.com/pybind/pybind11`, 2017.

[6] Wim T. L. P. Lavrijsen and Aditi Dutta. High-performance Python-C++ bindings with PyPy and Cling. *Proceedings of PyHPC*, 2016. INCOMPLETE: verify authors/venue.

[7] Bruno Lévy. Graphite and the Geogram Object Model (GOM). `https://github.com/BrunoLevy/GraphiteThree`. INCOMPLETE: prefer a paper if one exists — this is the closest prior art to the dynamic projection and deserves a proper citation, not a repository link.

[8] Daniel Sieger and Mario Botsch. The polygon mesh processing library. `https://github.com/pmp-library/pmp-library`, 2023. Version 3.0.0.

[9] The Qt Company. The Meta-Object system. Qt documentation. INCOMPLETE: cite a stable versioned URL.